

URL Shortener — Project Guide

A full-stack URL shortener built with the **PERN stack** (PostgreSQL, Express, React, Node.js) and Redis for caching. Paste a long URL, get a short link, and visiting that link redirects to the original destination.

Table of Contents

- 1. [Overview](#)
 - 2. [Features](#)
 - 3. [Architecture](#)
 - 4. [Prerequisites](#)
 - 5. [Quick Start \(Docker\)](#)
 - 6. [Manual Setup — PostgreSQL](#)
 - 7. [Manual Setup — Redis](#)
 - 8. [Backend Setup](#)
 - 9. [Frontend Setup](#)
 - 10. [API Reference](#)
 - 11. [Project Structure](#)
 - 12. [Environment Variables](#)
 - 13. [Troubleshooting](#)
-

1. Overview

Layer	Technology
Frontend	React 18 + Vite
Backend	Node.js + Express
Database	PostgreSQL 16
Cache	Redis 7
Infrastructure	Docker Compose

Default ports:

Service	Port
Frontend (Vite)	3000
Backend (Express)	5001
PostgreSQL	5432
Redis	6379

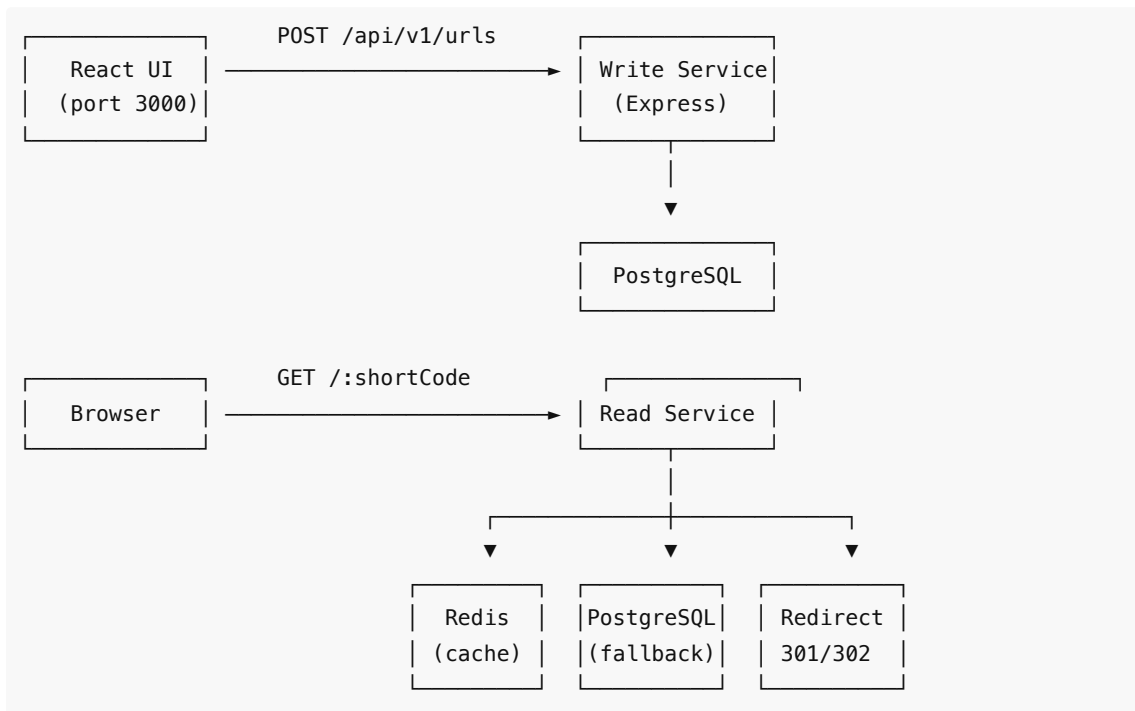
Note: Port 5001 is used instead of 5000 because macOS reserves port 5000 for AirPlay Receiver.

2. Features

- **Shorten URLs** — Submit a long URL and receive a compact short link

- **Redirect** — Visiting a short URL redirects to the original (301 permanent, 302 if expiring)
- **Custom alias** — Optionally choose your own path (e.g. `short/1club`)
- **Expiry** — Optionally set an expiration date; expired links return HTTP 410
- **Uniqueness** — PostgreSQL unique constraint ensures no two links share the same code
- **Caching** — Redis caches lookups for fast redirects; app works without Redis if unavailable

3. Architecture



Auto-generated short codes: PostgreSQL `BIGSERIAL` ID → Base62 encoding (e.g. ID `1` → `1` , ID `62` → `10`).

Custom aliases: Inserted directly with a database unique constraint; conflicts return HTTP 409.

4. Prerequisites

Install the following before running the project:

Tool	Minimum Version	Purpose
Node.js	18+	Runtime for backend and frontend
npm	9+	Package manager (bundled with Node)
Docker Desktop	Latest	Run PostgreSQL and Redis locally

Verify installations:

```

node -v    # v18.0.0 or higher
npm -v     # 9.0.0 or higher
  
```

```
docker -v # Docker version 20+
docker compose version
```

5. Quick Start (Docker)

This is the recommended way to run the project. Docker Compose starts PostgreSQL and Redis in containers.

Step 1: Clone and install dependencies

```
cd URL-Shortener
npm run install:all
```

Step 2: Start Docker Desktop

Open **Docker Desktop** and wait until it shows "Docker Desktop is running".

Step 3: Start PostgreSQL and Redis

```
npm run docker:up
```

This runs `docker compose up -d` and starts:

- **postgres** — PostgreSQL 16 on port 5432
- **redis** — Redis 7 on port 6379

Verify containers are running:

```
docker compose ps
```

Expected output:

NAME	STATUS	PORTS
url-shortener-postgres-1	running	0.0.0.0:5432->5432/tcp
url-shortener-redis-1	running	0.0.0.0:6379->6379/tcp

Step 4: Configure environment

Copy the example env file (already done if you cloned the repo):

```
cp backend/.env.example backend/.env
```

Default values work with Docker Compose out of the box:

```
PORT=5001
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/urlshortener
REDIS_URL=redis://localhost:6379
```

```
BASE_URL=http://localhost:5001
NODE_ENV=development
```

Step 5: Run database migration

Creates the `urls` table and indexes:

```
npm run db:migrate
```

Expected output: `Database migration complete`

Step 6: Start the backend

```
npm run dev:backend
```

Expected output:

```
Redis connected
Server running on http://localhost:5001
```

Step 7: Start the frontend (new terminal)

```
npm run dev:frontend
```

Open <http://localhost:3000> in your browser.

Step 8: Test it

1. Enter a long URL in the form (e.g. `https://google.com`)
2. Click **Shorten URL**
3. Copy the short link (e.g. `http://localhost:5001/1`)
4. Open the short link — you should be redirected to Google

Stop services

```
# Stop backend/frontend with Ctrl+C in their terminals

# Stop Docker containers
npm run docker:down
```

6. Manual Setup — PostgreSQL

Use this if you prefer a local PostgreSQL installation instead of Docker.

macOS (Homebrew)

```
# Install
brew install postgresql@16
```

```
# Start service
brew services start postgresql@16

# Create database
createdb urlshortener
```

Ubuntu / Debian

```
sudo apt update
sudo apt install postgresql postgresql-contrib

sudo systemctl start postgresql
sudo systemctl enable postgresql

# Create database and user
sudo -u postgres psql -c "CREATE DATABASE urlshortener;"
sudo -u postgres psql -c "CREATE USER postgres WITH PASSWORD 'postgres';"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE urlshortener TO
postgres;"
```

Windows

1. Download the installer from [postgresql.org/download/windows](https://www.postgresql.org/download/windows)
2. Run the installer; note the password you set for the `postgres` user
3. Open **pgAdmin** or **psql** and create a database named `urlshortener`

Update `.env`

Set `DATABASE_URL` to match your local credentials:

```
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/urlshortener
```

Then run the migration:

```
npm run db:migrate
```

Verify connection

```
psql postgresql://postgres:postgres@localhost:5432/urlshortener -c "\dt"
```

You should see the `urls` table listed.

7. Manual Setup — Redis

Use this if you prefer a local Redis installation instead of Docker.

macOS (Homebrew)

```
brew install redis
brew services start redis
```

Ubuntu / Debian

```
sudo apt update
sudo apt install redis-server

sudo systemctl start redis-server
sudo systemctl enable redis-server
```

Windows

Use [Memurai](#) (Redis-compatible) or run Redis via Docker:

```
docker run -d --name redis -p 6379:6379 redis:7-alpine
```

Verify connection

```
redis-cli ping
```

Expected response: `PONG`

Update `.env`

```
REDIS_URL=redis://localhost:6379
```

The backend connects to Redis on startup. If Redis is unavailable, the app still runs but skips caching (redirects fall back to PostgreSQL).

8. Backend Setup

Install dependencies

```
cd backend
npm install
```

Environment file

```
cp .env.example .env
```

Edit `.env` if your PostgreSQL or Redis connection details differ from the defaults.

Run migration

```
npm run db:migrate
```

Start development server

```
npm run dev
```

Uses Node.js `--watch` for automatic restarts on file changes.

Production start

```
npm start
```

Health check

```
curl http://localhost:5001/health
```

Response: `{"status":"ok"}`

9. Frontend Setup

Install dependencies

```
cd frontend  
npm install
```

Start development server

```
npm run dev
```

Runs on <http://localhost:3000>. API requests to `/api/*` are proxied to the backend at `http://localhost:5001` (configured in `vite.config.js`).

Production build

```
npm run build  
npm run preview
```

10. API Reference

Create short URL

POST /api/v1/urls
Content-Type: application/json

Request body:

```
{
  "long_url": "https://example.com/very/long/path",
  "custom_alias": "my-link",
  "expires_at": "2027-01-01T00:00:00Z"
}
```

Field	Required	Description
long_url	Yes	Must be http:// or https://, max 2048 chars
custom_alias	No	3–20 alphanumeric chars, hyphens, or underscores
expires_at	No	ISO 8601 datetime in the future; omit for no expiry

Success — 201 Created:

```
{
  "short_url": "http://localhost:5001/my-link",
  "short_code": "my-link",
  "long_url": "https://example.com/very/long/path",
  "expires_at": "2027-01-01T00:00:00.000Z"
}
```

Errors:

Status	Meaning
400	Invalid URL, alias, or expiry
409	Custom alias already taken

Example with curl:

```
# Auto-generated code
curl -X POST http://localhost:5001/api/v1/urls \
  -H "Content-Type: application/json" \
  -d '{"long_url": "https://google.com"}'

# Custom alias
curl -X POST http://localhost:5001/api/v1/urls \
  -H "Content-Type: application/json" \
  -d '{"long_url": "https://google.com", "custom_alias": "1club"}'
```

Redirect

```
GET /:shortCode
```

Status	Meaning
301	Permanent redirect (no expiry set)
302	Temporary redirect (expiry set)
404	Short code not found
410	Link has expired

Example:

```
curl -I http://localhost:5001/1club
```

Health check

```
GET /health
```

Response: `{"status":"ok"}`

11. Project Structure

```
URL-Shortener/
├── backend/
│   ├── src/
│   │   ├── index.js           # Entry point
│   │   ├── app.js             # Express app setup
│   │   ├── config/
│   │   │   ├── database.js    # PostgreSQL pool
│   │   │   └── redis.js       # Redis client
│   │   ├── controllers/
│   │   │   └── urlController.js # Request handlers
│   │   ├── db/
│   │   │   ├── schema.sql     # Table definition
│   │   │   └── migrate.js     # Migration runner
│   │   ├── middleware/
│   │   │   ├── validateCreateUrl.js
│   │   │   └── errorHandler.js
│   │   ├── routes/
│   │   │   └── urlRoutes.js
│   │   ├── services/
│   │   │   └── urlService.js   # Business logic + cache
│   │   └── utils/
│   │       ├── base62.js      # ID → short code encoding
│   │       └── validateUrl.js  # URL and alias validation
│   ├── .env.example
│   └── package.json
```

```

├── frontend/
│   ├── src/
│   │   ├── App.jsx          # Main layout
│   │   ├── App.css          # Styles
│   │   ├── main.jsx         # React entry
│   │   ├── api/
│   │   │   ├── client.js    # API calls
│   │   │   └── components/
│   │   │       └── UrlForm.jsx # URL submission form
│   ├── index.html
│   ├── vite.config.js
│   └── package.json
├── docs/
│   └── GUIDE.md              # This guide
├── docker-compose.yml        # PostgreSQL + Redis
└── package.json              # Root scripts

```

12. Environment Variables

Variable	Default	Description
PORT	5001	Backend server port
DATABASE_URL	postgresql://postgres:postgres@localhost:5432/urlshortener	PostgreSQL connection string
REDIS_URL	redis://localhost:6379	Redis connection string
BASE_URL	http://localhost:5001	Base URL used in generated short links
NODE_ENV	development	Environment mode

13. Troubleshooting

Port 5000 already in use (EADDRINUSE)

On macOS, **AirPlay Receiver** uses port 5000. This project uses port **5001** by default. If you still see this error, check what's using the port:

```
lsof -i :5001
```

Kill the conflicting process or change `PORT` in `backend/.env`.

To free port 5000 on Mac: **System Settings** → **General** → **AirDrop & Handoff** → **AirPlay Receiver** → **Off**.

Docker: "Cannot connect to Docker daemon"

Start **Docker Desktop** and wait until it is fully running, then retry:

```
npm run docker:up
```

Database connection refused

1. Confirm PostgreSQL is running:

```
docker compose ps
# or
brew services list | grep postgres
```

2. Verify `DATABASE_URL` in `backend/.env` matches your credentials.
3. Run migration if you haven't:

```
npm run db:migrate
```

Redis connection warning

The app logs `Redis unavailable, running without cache` if Redis is down. Redirects still work via PostgreSQL, just without caching.

Start Redis:

```
npm run docker:up
# or
brew services start redis
```

Frontend can't reach API

Ensure the backend is running on port 5001. The Vite dev server proxies `/api` to `http://localhost:5001` — if you change the backend port, update `frontend/vite.config.js` too.

Custom alias returns 409

That alias is already taken. Choose a different one.

Short URL returns 404

The code doesn't exist. Confirm you created it first and you're using the backend URL (`http://localhost:5001/CODE`), not the frontend URL.

Database Schema

```
CREATE TABLE urls (
  id          BIGSERIAL PRIMARY KEY,
  short_code  VARCHAR(20) NOT NULL UNIQUE,
  long_url    TEXT NOT NULL,
  expires_at  TIMESTAMPTZ,
```

```
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Generated for URL Shortener — PERN Stack